

Racetrack API Reference

Overview

The Racetrack API exposes a read-only, frame-aware view over the procedurally generated track used by the Game AI racetrack assignments. It provides initialization state, scalar properties, absolute and relative arc-length sampling, nearest-point queries, cached vertex-path data, Bézier control data, and small helpers for mapping and enumeration. All members are read-only.

All data are provided in world-space coordinates and are refreshed once per frame to reflect the current state of the procedurally generated track.

The track geometry is represented internally in two forms: the original cubic Bézier chain and a tessellated vertex path (VertexPath). Direct evaluation of cubic Bézier curves for properties such as arc length, distance-based sampling, or nearest-point queries is computationally expensive or impractical to perform in real time. To address this, the Bézier chain is tessellated into the VertexPath, which provides an efficient, piecewise-linear approximation of the track centerline. This tessellated form enables fast and accurate distance-based sampling, is used for generating physics colliders, and supports efficient rendering. For both performance and accuracy reasons, the Racetrack API exposes this tessellated VertexPath for most queries, including position and direction sampling, nearest-point queries, and cached vertex-path data. The Bézier chain representation is primarily exposed for advanced geometry diagnostics and access to the original control points.

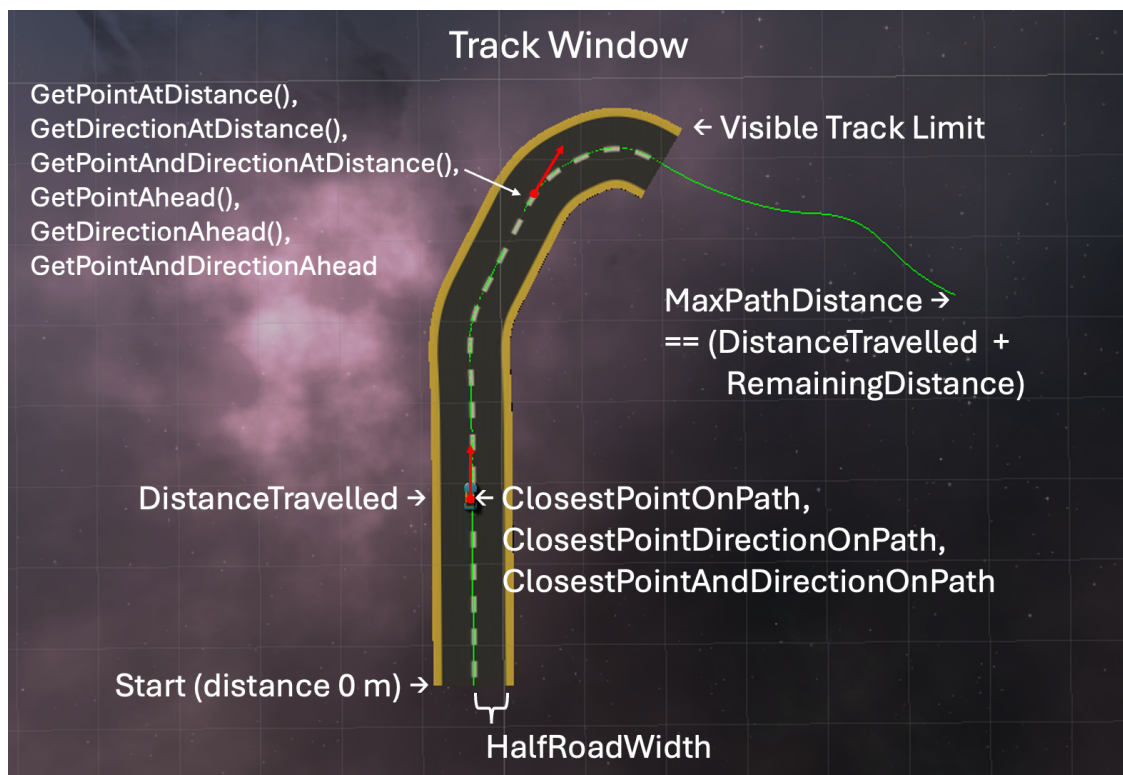


Figure 1: Example showing racetrack distance and related properties along an active track window.

Credits

The procedurally generated track system is derived from Seb Lague’s *Path-Creator*, a Bézier curve and path editing tool for Unity (<https://github.com/SebLague/Path-Creator>). The implementation used here has

been significantly modified to support real-time procedural generation and runtime sampling within the Racetrack API.

Initialization and Core Properties

Racetrack.IsInitialized

True when the underlying dependencies have been initialized and a valid path exists. Always guard sampling with this before use.

Racetrack.HalfRoadWidth

The constant half-width of the road from its centerline. This may return a default or placeholder value until the path has been fully initialized.

Racetrack.DistanceTravelled

Distance (meters) along the currently active track window. This is local to the active procedural segment set.

Racetrack.TotalDistanceTravelled

Total forward distance (meters) since race start.

Racetrack.MaxPathDistance

Full arc-length (meters) of the current active track window. Clamp look-ahead to this to avoid out-of-range queries.

Racetrack.RemainingDistance

Convenience: distance remaining in the current track window.

Racetrack.ClosestPointOnPath

World position on the centerline closest to the vehicle.

Racetrack.ClosestPointDirectionOnPath

Path tangent (direction) at the closest point.

Arc-length queries (absolute s in meters)

```
Vector3 Racetrack.GetPointAtDistance(float s)
Vector3 Racetrack.GetDirectionAtDistance(float s)
void    Racetrack.GetPointAndDirectionAtDistance(float s,
                                                    out Vector3 point,
                                                    out Vector3 direction)
```

Sample position/tangent at absolute distance s. Use `Racetrack.ClampDistance(s)` if you are not sure s is in range.

Relative look-ahead (offset from vehicle position)

```
Vector3 Racetrack.GetPointAhead(float offsetMeters)
Vector3 Racetrack.GetDirectionAhead(float offsetMeters)
void Racetrack.GetPointAndDirectionAhead(float offsetMeters,
                                         out Vector3 point,
                                         out Vector3 direction)
```

Sample ahead of the car by offsetMeters along the centerline.

Closest-point queries for arbitrary world positions

```
Vector3 Racetrack.GetClosestPointOnPath(Vector3 worldPos)
Vector3 Racetrack.GetClosestPointDirectionOnPath(Vector3 worldPos)
void Racetrack.GetClosestPointAndDirectionOnPath(Vector3 worldPos,
                                                  out Vector3 point,
                                                  out Vector3 direction)
```

Retrieve nearest centerline point and tangent for any worldPos.

Vertex-path cache (read-only, refreshed automatically)

These cached data come from the tessellated VertexPath representation of the track, which is a discretized approximation of the track centerline used for most runtime queries.

```
ICollection<Vector3> Racetrack.Vertices
ICollection<Vector3> Racetrack.Tangents
ICollection<float> Racetrack.CumulativeLength
```

Cached discretization of the active path. Use for custom geometry, curvature, or diagnostics; values update as the track regenerates.

Index mapping and enumeration helpers

```
bool Racetrack.TryMapDistanceToVertexIndex(float s, out int index)
bool Racetrack.TryMapWindowToVertexRange(float s0, float s1,
                                         out int i0, out int i1)

IEnumerable<(float s, Vector3 point, Vector3 tangent)>
    Racetrack.EnumerateWindow(float s0, float s1, int stride = 1)

bool Racetrack.TryGetVertexSample(int index,
                                  out float s,
                                  out Vector3 point,
                                  out Vector3 tangent)

IEnumerable<(int index, float s, Vector3 point, Vector3 tangent)>
    Racetrack.EnumerateWindowIndexed(float s0, float s1, int stride = 1)
```

Maps arc-lengths to cached vertex indices and iterates cached samples. Prefer the indexed form when using stride > 1 or when you need indices for external bookkeeping.

Bézier control data (read-only, world space)

These are the original control points (transformed into world space) from the procedural cubic Bézier chain that defines the track geometry.

```
int Racetrack.BezierSegmentCount
IReadOnlyList<Vector3> Racetrack.BezierP0
IReadOnlyList<Vector3> Racetrack.BezierP1
IReadOnlyList<Vector3> Racetrack.BezierP2
IReadOnlyList<Vector3> Racetrack.BezierP3

bool Racetrack.TryGetClosestBezierSegmentSummary(out int segIndex,
                                                  out Vector3 p0,
                                                  out Vector3 p1,
                                                  out Vector3 p2,
                                                  out Vector3 p3)
```

Per-segment control points for the cubic Bézier chain that underlies the vertex path, transformed into world space. The summary copies the four world-space control points for the segment nearest the vehicle.

Utility

```
float Racetrack.ClampDistance(float s)
```

Clamps an arc-length to the active window [0, MaxPathDistance]. Many sampling methods use this bound internally to ensure safe queries.

Indices

```
int Racetrack.CurrentBezierSegmentIndex
int Racetrack.CurrentClosestPathPointIndex
```

Indices of the currently nearest Bézier segment and nearest vertex, respectively. CurrentClosestPathPointIndex refers to the tessellated vertex path, while CurrentBezierSegmentIndex refers to the Bézier chain segment. These pair naturally with the Bézier control arrays and the cached vertex arrays documented above.

Other useful Unity helpers

```
float Vector3.SignedAngle(Vector3 from, Vector3 to, Vector3 axis)
float Mathf.Sign(float x)
Vector3 Vector3.Project(Vector3 vector, Vector3 onNormal)
```

SignedAngle finds the signed angle between two vectors; Sign returns the sign of a float (-1f or 1f).

Project can be used to project a vector onto a plane or axis. Note: The racecar's position is slightly above the ground due to the tires, and the racetrack lies in the XZ plane. To ensure computations (such as nearest-point queries) are aligned with the track, it's recommended to project the vehicle's position downward onto the XZ plane before processing.